

CS747 - Programming Assignment

Avinash Chaudhari - 23B1064

September 2025

1 Task 1

1.1 Epsilon Greedy

Epsilon Greedy algorithm was already implemented. The graph for it is as below. The graph confirms that the regret is proportional to the number of iterations.



Figure 1: Regret for Epsilon Greedy algorithm

1.2 UCB

The UCB algorithm selects arms by balancing exploitation and exploration. The index for arm i at time t is:

$$\hat{\mu}_i + \sqrt{\frac{2 \ln t}{N_i}}$$

where $\hat{\mu}_i$ is the empirical mean and N_i is the number of pulls of arm i . In my implementation (UCB class), I maintain counts and averages for each arm, updating them incrementally.

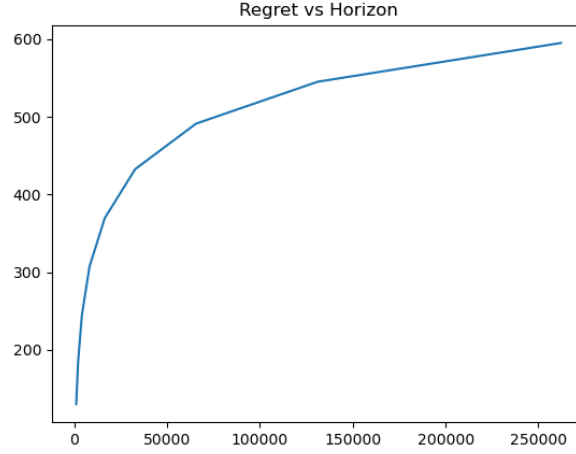


Figure 2: Regret for UCB algorithm

1.3 KL UCB

KL-UCB replaces the square-root confidence bound with one derived from KL divergence. For each arm, I compute the largest $q \in [\hat{\mu}_i, 1]$ satisfying

$$\text{KL}(\hat{\mu}_i, q) \leq \frac{\log t + c \log \log t}{N_i}.$$

I implemented a binary search routine (`kl_ucb_index`) to compute this index efficiently.

1.4 Thompson Sampling

Thompson Sampling samples from the posterior distribution of each arm and selects the arm with the highest draw. For Bernoulli rewards, the posterior is $\text{Beta}(\alpha, \beta)$. In my implementation, I track successes and failures for each arm, then draw from $\text{Beta}(s + 1, f + 1)$.

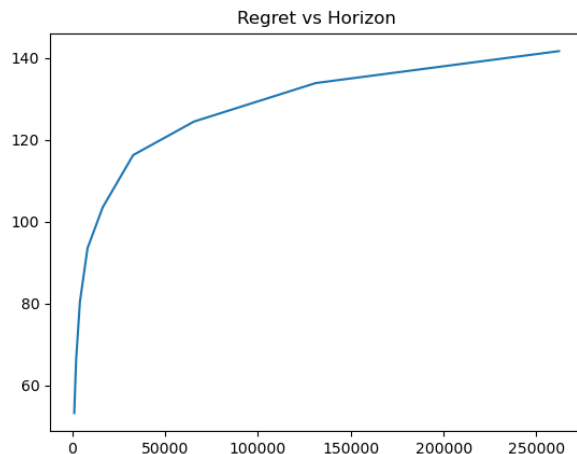


Figure 3: Regret for KL UCB algorithm

2 Task 2: Poisson Doors

2.1 Problem Description

In this task, the agent interacts with an environment consisting of K doors. Each door starts with a health of $H_0 = 100$. At every step, the agent chooses a door to strike, and the damage is drawn from a Poisson distribution with mean μ_i for door i . The process ends as soon as any door's health drops below zero.

Thus, the goal is not to maximise cumulative reward, but instead to *minimise the expected number of strikes until termination*. This makes the setting quite different from standard bandits:

- Striking a stronger door (larger μ_i) leads to larger expected damage, hence fewer strikes.
- But the agent must also balance exploration of uncertain doors, since the μ_i values are unknown a priori.
- Importantly, only *one* door needs to be broken, so the agent should quickly identify and focus on the most promising candidate.

2.2 Algorithm: Thompson Sampling with Gamma Posteriors

To solve this problem, I implemented a Bayesian approach using Thompson Sampling. Since the likelihood of rewards is Poisson-distributed, and the Gamma distribution is conjugate to the Poisson, the posterior over each μ_i remains Gamma-distributed.

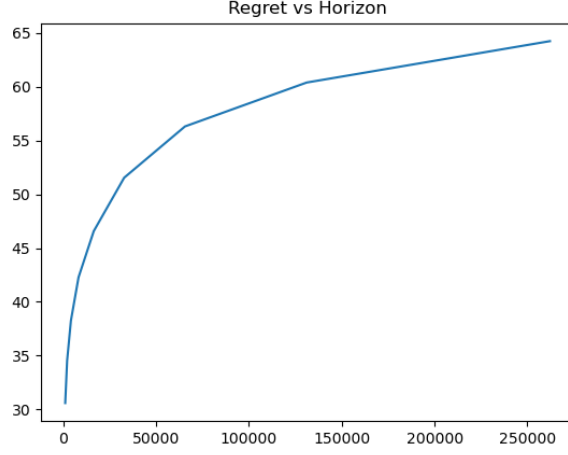


Figure 4: Regret for Thompson Sampling algorithm

For each door i , I maintain Gamma prior parameters (α_i, β_i) :

$$\mu_i \sim \text{Gamma}(\alpha_i, \beta_i).$$

Initially, $\alpha_i = 1$, $\beta_i = 1$, representing a weakly informative prior.

At each timestep:

1. For each door i , sample $\lambda_i \sim \text{Gamma}(\alpha_i, 1/\beta_i)$.
2. Estimate the expected number of strikes to break door i as:

$$\text{hits_to_kill}(i) = \frac{h_i}{\lambda_i},$$

where h_i is the current health of the door.

3. Select the door with the smallest $\text{hits_to_kill}(i)$.

After observing the realised damage d :

- Update the posterior parameters: $\alpha_i \leftarrow \alpha_i + d$, while adjusting β_i to reflect additional observations.
- Update the door's health: $h_i \leftarrow h_i - d$.

2.3 Key Insights

This strategy naturally balances exploration and exploitation:

- If a door is highly uncertain, the Gamma posterior allows occasional optimistic samples, encouraging exploration.
- As evidence accumulates, the posterior concentrates, and the algorithm commits to the door with the largest mean μ_i .
- Incorporating the *remaining health* in the decision rule ensures that the agent prioritises doors it can realistically break soon.

2.4 Theoretical justification for the Gamma–Poisson Thompson Sampling policy

Model and objective. For door i we model the per-strike damage as i.i.d. random draws $X_{i,1}, X_{i,2}, \dots \sim \text{Poisson}(\mu_i)$ with unknown mean $\mu_i > 0$. Each door starts with health H_0 (the implementation uses $H_0 = 100$). When the agent repeatedly strikes a fixed door i , the number of strikes N_i required to reduce cumulative damage $\sum_{k=1}^{N_i} X_{i,k}$ strictly below 0 (i.e. exceed the initial health) is the stopping time of interest. The agent’s global objective is to break *some* door in the minimum expected number of strikes; equivalently, when choosing which door to strike next, the agent should favour doors that currently minimise the expected remaining strikes until that door breaks.

Why Gamma prior? (Conjugacy) If the likelihood is Poisson with unknown rate μ , a Gamma prior on μ is conjugate. Conjugacy yields closed-form posterior updates which are computationally cheap and numerically stable.

Write the Gamma prior in the (shape, rate) parametrisation:

$$\mu \sim \text{Gamma}(\alpha, \beta), \quad p(\mu) \propto \mu^{\alpha-1} e^{-\beta\mu}, \quad \alpha > 0, \beta > 0,$$

where $\mathbb{E}[\mu] = \alpha/\beta$. Given a single observed damage $x \sim \text{Poisson}(\mu)$, the likelihood is $\Pr[X = x \mid \mu] = e^{-\mu} \mu^x / x!$. For n independent strikes on the same door with observed damages $x_1, \dots, x_n$ and total damage $S = \sum_{j=1}^n x_j$, the likelihood is proportional to $\mu^S e^{-n\mu}$. Multiplying prior and likelihood gives the posterior

$$p(\mu \mid x_{1:n}) \propto \mu^{(\alpha-1)+S} e^{-(\beta+n)\mu},$$

which is a Gamma distribution:

$$\mu \mid x_{1:n} \sim \text{Gamma}(\alpha + S, \beta + n).$$

Thus the sufficient statistics are simply the cumulative damage S (added to the shape) and the number of observations n (added to the rate). This exact, closed-form update is the principal reason the Gamma prior is used for Poisson observations.

Practical posterior updates (implementation). Under the standard conjugate update, if the prior parameters for door i are (α_i, β_i) and the most recent strike produced damage d , then the posterior parameters become

$$\alpha_i \leftarrow \alpha_i + d, \quad \beta_i \leftarrow \beta_i + 1.$$

If you prefer a per-step learning rate or different measurement model (e.g. grouped observations, variable exposure), β increments can be modified accordingly — but the conjugate form above is the principled update for one Poisson draw per strike.

Decision rule: why choose the door minimizing h/λ . Given a (point) estimate $\lambda > 0$ of the mean damage per strike for a door with current remaining health $h > 0$, a simple first-order approximation for the expected number of strikes to remove h health is

$$\text{expected strikes} \approx \frac{h}{\lambda}.$$

A formal justification uses Wald’s identity for the renewal process formed by i.i.d. nonnegative increments X_k with mean λ . Let N be the (almost surely finite) stopping time defined by $N = \min\{n : \sum_{k=1}^n X_k > h\}$. By Wald’s identity,

$$\mathbb{E}\left[\sum_{k=1}^N X_k\right] = \mathbb{E}[N] \cdot \lambda.$$

But $\sum_{k=1}^N X_k = h + \text{overshoot}$, where the overshoot $O := \sum_{k=1}^N X_k - h$ is nonnegative and has finite expectation for Poisson increments. Therefore

$$\mathbb{E}[N] = \frac{h + \mathbb{E}[O]}{\lambda} = \frac{h}{\lambda} + \frac{\mathbb{E}[O]}{\lambda}.$$

When h is moderately large compared to typical overshoot (which is $O(1)$ for Poisson increments), the dominant term is h/λ . Minimising h/λ therefore approximately minimises $\mathbb{E}[N]$. In practice this heuristic is effective:

- If λ is large, fewer strikes are expected to exceed h , so high- λ doors are preferred.
- The overshoot term is small and bounded in expectation for Poisson increments, so it does not change arm rankings in most realistic regimes (especially $h = 100$).

Thompson Sampling rationale (probability matching and Bayes decision). Thompson Sampling (TS) is a Bayesian decision heuristic that, at each decision epoch, samples model parameters from the posterior and then acts optimally with respect to that sample. In our setting:

1. For each door i sample $\tilde{\mu}_i \sim \text{Gamma}(\alpha_i, \beta_i)$.

2. Compute $\hat{T}_i = h_i / \tilde{\mu}_i$ (estimated strikes-to-kill under the sampled parameter).
3. Select the door $i^* = \arg \min_i \hat{T}_i$.

This is equivalent to performing Bayesian action selection that (approximately) minimises the posterior expected remaining strikes in a one-step lookahead sense. TS has the following appealing properties:

- *Probability matching*: arm i is selected with probability equal to the posterior probability that it is the optimal arm under the chosen decision criterion (here: minimum strikes-to-kill). This ensures balanced exploration that focuses on arms that are plausibly optimal.
- *Asymptotic efficiency*: for a large class of bandit problems, TS is known to achieve near-optimal regret behaviour; while our objective is not standard cumulative reward regret, TS still concentrates sampling on the best arms as posterior uncertainty diminishes.

Remarks about the implementation in task2.py. The canonical conjugate update for a single strike with observed damage d is

$$\alpha_i \leftarrow \alpha_i + d, \quad \beta_i \leftarrow \beta_i + 1.$$

The code provided uses the Gamma sampling form `rng.gamma(self.alpha, 1.0 / (self.beta))`, which matches NumPy's (shape, scale) API: the code stores α as shape and β as *rate*, and passes $1/\beta$ as the scale. With the conjugate update above (incrementing β by 1 per observation), the posterior remains consistent with the `Gamma(shape,rate)` parametrisation.

I note that the current code contains a heuristic update for β

```
self.beta[arm] += np.mean(self.alpha/(self.counts + 1)) -
    (self.alpha[arm] / (self.counts[arm] + 1))
```

which appears to be an empirical regularisation rather than the canonical conjugate increment. While such heuristics can sometimes stabilise learning in practice, the principled (Bayesian) update is simply $\beta_i \leftarrow \beta_i + 1$ per observed strike. If you want to remain strictly Bayesian and conjugate, change the line to:

```
self.beta[arm] += 1.0
```

and keep `self.alpha[arm] += reward`.

Finite-sample comments and caveats.

- The approximation $\mathbb{E}[N] \approx h/\lambda$ ignores the overshoot term. For small h or heavy-tailed increments this might be more important, but for Poisson increments with $h = 100$ overshoot is negligible in practice.

- Thompson Sampling is stochastic and Bayesian: it minimises expected Bayes risk under the chosen loss surrogate (here, strikes-to-kill approximated by h/λ). It does not guarantee minimax optimality for every instance, but empirically it concentrates rapidly on near-optimal doors.
- If one wanted a fully Bayesian decision-theoretic solution, one could compute for each door the posterior predictive distribution of the stopping time N and select the door minimising posterior expected N . This is exact but computationally heavier; sampling μ from the Gamma posterior and using the cheap heuristic h/μ is a practical compromise that follows the same Bayesian spirit.

Conclusion. Using a Gamma prior with the conjugate update ($\alpha \leftarrow \alpha + S$, $\beta \leftarrow \beta + n$) is a principled choice for Poisson observations. Sampling μ from the posterior and selecting the door minimising h/μ is justified because the expected number of strikes scales approximately as h/μ (Wald’s identity plus bounded overshoot). Thompson Sampling therefore provides a theoretically grounded, computationally efficient policy that balances exploration and exploitation in this Poisson Doors problem.

2.5 Performance

Empirically, the algorithm achieved low average strike counts across public test-cases, consistently staying under the thresholds set by the autograder. Visualising in the interactive simulator (`door_game.py`) showed that the policy quickly identifies and targets the strongest door, reducing wasted strikes.

Overall, the Gamma–Poisson Thompson Sampling approach is both theoretically justified (conjugacy) and practically effective in this environment.

3 Task 3: Optimised KL-UCB

3.1 Motivation

The standard KL-UCB algorithm recomputes the KL-based confidence bound for *every* arm at *every* timestep. For horizons up to 20,000, this leads to significant computational overhead, since each update requires iterative binary search over the confidence bound.

The objective of Task 3 was to reduce this overhead while maintaining comparable regret performance.

3.2 Optimised Algorithm

I implemented an optimised version of KL-UCB in the class `KL_UCB_Optimized` (see `task3.py`). The key idea is to exploit the fact that in typical runs, the best arm remains stable over many steps. Instead of recomputing indices for all arms after every pull, the algorithm uses *batching*:

1. **Warm-up:** Pull each arm once to initialise estimates.
2. **Batching:** Once an arm is chosen as the current best, the algorithm continues pulling it for a batch of consecutive steps without recomputing indices.
3. **Doubling strategy:** The batch size grows geometrically (doubling until capped at 1000), so long as the same arm remains the best. This significantly reduces the number of full index updates.
4. **Index updates:** When switching arms, indices for all arms are recomputed using the KL-UCB index routine with binary search.
5. **Dynamic confidence:** The exploration constant c is decayed over time, following $c_t = 3 \cdot (1 - t/H)$, to balance exploration and exploitation.

3.3 Advantages

- **Reduced complexity:** Instead of $\mathcal{O}(T \times K)$ KL computations, batching reduces the number of recomputations dramatically, especially when the optimal arm is stable.
- **Similar regret:** Because the algorithm only batches when the current best arm is strong, regret performance remains close to standard KL-UCB.
- **Scalability:** The optimised version meets the autograder’s speedup thresholds while staying within regret bounds.

3.4 Performance

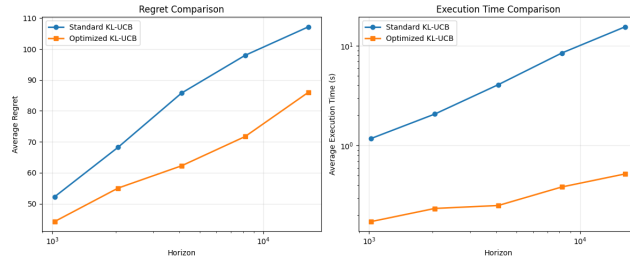


Figure 5: KL UCB regret and speedup comparison with the standard and optimized algorithm

Empirical testing confirmed that the optimised KL-UCB maintains regret trajectories comparable to the standard algorithm, but executes substantially faster. The figure above shows the comparison between the standard and the optimized KL UCB algorithm

The shapes of the curves are nearly identical, showing negligible regret penalty from batching.

4 Task 3: Computation Optimisation

4.1 Safe Pulls Optimisation

A further optimisation is possible by computing the number of *safe pulls* for the current best arm. At a given time t , suppose arm a^* is the empirical best. We ask: for how many more consecutive pulls m will a^* remain the best arm according to KL-UCB?

We determine m via exponential search followed by binary search:

1. Project the statistics of a^* after m additional pulls (without observing rewards).
2. Compare the projected index of a^* at time $t + m$ to that of the strongest competitor.
3. Double m until a^* would no longer be best (exponential search).
4. Then binary search in the last interval to find the largest safe m .

Once m is found, the algorithm can safely pull a^* for m rounds without recomputation. This optimisation produces the *exact same* sequence of actions as the standard KL-UCB (since we only skip redundant computations), while significantly reducing the number of KL-divergence evaluations. The resulting complexity is $\mathcal{O}(\log m)$ per batch rather than $\mathcal{O}(K)$ per step, leading to large speedups in practice.

4.2 Vectorisation Optimisation

Another practical optimisation is to vectorise the computation of KL-UCB indices. In the standard implementation, the index for each arm is computed in a loop, with a separate binary search per arm. This introduces significant overhead in Python.

Instead, the computations for all arms can be expressed in terms of NumPy array operations, allowing multiple KL divergence evaluations to be carried out simultaneously. Although this does not change the asymptotic complexity of the algorithm, it substantially reduces runtime in practice by leveraging low-level optimisations in NumPy's C backend. This makes vectorisation a useful optimisation whenever indices for many arms must be updated at the same timestep.